

人工智能基础实验一

实验报告

实验题目：编程解决野人与传教士问题

专 业：人工智能

姓 名：马承乾

学 号：22920212204180

实验日期：2023.10.26

一. 实验题目

1 什么是野人与传教士问题：

在河的左岸有 N 个传教士、 N 个野人和一条船，传教士们想用这条船把所有的人都运过河去，但有以下条件限制：（设 $N=3$ ）

- （1）修道士和野人都会划船，但船每次最多只能运 K 个人（设 $K=2$ ）；
- （2）在任何岸边野人数目都不能超过修道士，否则修道士会被野人吃掉。

假定野人会服从任何一种过河安排，请规划出一个确保修道士安全过河的计划。

二. 实验目的

- 1. 理解野人与传教士问题的解题思路
- 2. 编程实现该解题思路
- 3. 得到问题的最终答案

三. 实验方法

1 将题目条件数学化

（1） $M \geq C$ 任何时刻两岸、船上都必须满足传教士人数不少于野人数（ $M=0$ 时除外，既没有传教士）

（2） $M+C \leq K$ 船上人数限制在 K 以内

2 具体实现思路：

设计算法思路如下：

使用深度优先搜索 dfs 和数据结构双端队列 (deque) 来解决传教士和野人问题：

使用全局变量 num_of_resolution 统计解法数；

定义队列 q 用于保存每一步的状态 (传教士数, 野人数)。

定义 Print 函数打印每一个解法, 将队列元素打印出来。

定义 missionaries_and_cannibals 函数定义搜索状态空间的递归过程:

函数共有四个参数, m 和 c 表示传教士和野人数目, q 表示存储过河方案的过程的队列, side 表示当前在哪一岸, side=0 时表明当前在左岸, 也就是最初所在河岸; side=1 表示当前在右岸, 也就是最终要到达的河岸。

检查状态是否合法(人数是否小于 0), 如果达到目标状态(传教士和野人全过河), 则打印解法;

否则, 通过双重循环尝试不同人数的过河组合, 过河组合要注意以下内容:

- 2.1 限定船上最多 2 人, 过滤不合法或重复的过河方案, 用三维数组 visit 记录当前状态是否已访问。
- 2.2 如果这次过河是安全的, 则将新状态加入队列递归搜索, 搜索结束后从队列弹出这个状态。
- 2.3 永远不能空船, 否则谁来开船?

以 (3, 3, 0) 的初始状态开始递归搜索。

这样利用队列存储每个层次的状态, 递归调用实现深度优先搜索, 找出所有可能的解法。

实验代码见附件。

四、实验结果

实验结果如下:

第 1 个方案

当前船在左岸, 左岸有 3 个传教士和 3 个野人
当前船在右岸, 左岸有 3 个传教士和 1 个野人
当前船在左岸, 左岸有 3 个传教士和 2 个野人
当前船在右岸, 左岸有 3 个传教士和 0 个野人
当前船在左岸, 左岸有 3 个传教士和 1 个野人
当前船在右岸, 左岸有 1 个传教士和 1 个野人
当前船在左岸, 左岸有 2 个传教士和 2 个野人
当前船在右岸, 左岸有 0 个传教士和 2 个野人
当前船在左岸, 左岸有 0 个传教士和 3 个野人
当前船在右岸, 左岸有 0 个传教士和 1 个野人
当前船在左岸, 左岸有 0 个传教士和 2 个野人
当前船在右岸, 左岸有 0 个传教士和 0 个野人

第 2 个方案

当前船在左岸, 左岸有 3 个传教士和 3 个野人
当前船在右岸, 左岸有 3 个传教士和 1 个野人
当前船在左岸, 左岸有 3 个传教士和 2 个野人
当前船在右岸, 左岸有 3 个传教士和 0 个野人
当前船在左岸, 左岸有 3 个传教士和 1 个野人
当前船在右岸, 左岸有 1 个传教士和 1 个野人
当前船在左岸, 左岸有 2 个传教士和 2 个野人
当前船在右岸, 左岸有 0 个传教士和 2 个野人
当前船在左岸, 左岸有 0 个传教士和 3 个野人
当前船在右岸, 左岸有 0 个传教士和 1 个野人
当前船在左岸, 左岸有 1 个传教士和 1 个野人
当前船在右岸, 左岸有 0 个传教士和 0 个野人

第 3 个方案

当前船在左岸，左岸有 3 个传教士和 3 个野人
当前船在右岸，左岸有 2 个传教士和 2 个野人
当前船在左岸，左岸有 3 个传教士和 2 个野人
当前船在右岸，左岸有 3 个传教士和 0 个野人
当前船在左岸，左岸有 3 个传教士和 1 个野人
当前船在右岸，左岸有 1 个传教士和 1 个野人
当前船在左岸，左岸有 2 个传教士和 2 个野人
当前船在右岸，左岸有 0 个传教士和 2 个野人
当前船在左岸，左岸有 0 个传教士和 3 个野人
当前船在右岸，左岸有 0 个传教士和 1 个野人
当前船在左岸，左岸有 0 个传教士和 2 个野人
当前船在右岸，左岸有 0 个传教士和 0 个野人

第 4 个方案

当前船在左岸，左岸有 3 个传教士和 3 个野人
当前船在右岸，左岸有 2 个传教士和 2 个野人
当前船在左岸，左岸有 3 个传教士和 2 个野人
当前船在右岸，左岸有 3 个传教士和 0 个野人
当前船在左岸，左岸有 3 个传教士和 1 个野人
当前船在右岸，左岸有 1 个传教士和 1 个野人
当前船在左岸，左岸有 2 个传教士和 2 个野人
当前船在右岸，左岸有 0 个传教士和 2 个野人
当前船在左岸，左岸有 0 个传教士和 3 个野人
当前船在右岸，左岸有 0 个传教士和 1 个野人
当前船在左岸，左岸有 1 个传教士和 1 个野人
当前船在右岸，左岸有 0 个传教士和 0 个野人

五. 实验分析

通过本次实验，我亲自动手变成完成了野人与传教士问题的求解，在此过程中，我对深度优先搜索算法和 deque 数据结构的使用有了更深刻的认识。

六. 附录

实验代码（含注释）：

```
from collections import deque

num_of_resolution = 0

visit = [[[0] * 2 for _ in range(4)] for _ in range(4)]

q = deque()
def Print(q):
    global num_of_resolution
    num_of_resolution += 1
    print('第 %d 个方案' % num_of_resolution)
    print('当前船在左岸，左岸有 3 个传教士和 3 个野人')
    len_double_queue = len(q)
    for i in range(len_double_queue):
        j = q.popleft()# 左边出队
        if j[2] == 0:
            print('当前船在左岸，左岸有%d 个传教士和%d 个野人'%(j[0], j[1]))
        else:
            print('当前船在右岸，左岸有%d 个传教士和%d 个野人'%(3-j[0], 3-
j[1]))
        q.append(j)# 插回去

def missionaries_and_cannibals(m, c, side, q):
```

```

global visit
if visit[m][c][side] == 1:
    return
visit[m][c][side] = 1
if m > 3 or c > 3:
    return
elif m < 0 or c < 0:# 非法状态
    return
elif m < c and m != 0:# 传教士被吃了
    return
elif m > c and m != 3:# 对面的传教士被吃了
    return
elif m == 3 and c == 3 and side == 1:# 全过河了
    Print(q)
elif m >= c or m == 3 or m == 0:# 传教士能活
    for num_cross_river_m in range(m + 1):# 过河传教士
        for num_cross_river_c in range(c + 1):# 过河野人
            if num_cross_river_c + num_cross_river_m > 2:
                continue# 人别太多啊，别超过 2 人
            elif num_cross_river_m == 0 and num_cross_river_c == 0:
                continue# 没人别开船
            elif num_cross_river_m >= num_cross_river_c or
num_cross_river_c == 0 or num_cross_river_m == 0:# 船上野人也吃人
                q.append((3 - (m - num_cross_river_m), 3 - (c -
num_cross_river_c), 1 - side))
                missionaries_and_cannibals(3 - (m -
num_cross_river_m), 3 - (c - num_cross_river_c), 1 - side, q)
                q.pop()
            visit[m][c][side] = 0

missionaries_and_cannibals(3, 3, 0, q)
print('共有%d 个方案' % num_of_resolution)

```